

UNIVERSIDAD NACIONAL DE COLOMBIA
FACULTAD DE INGENIERÍA - SEDE BOGOTÁ



Facultad de Ingeniería
Informe Taller 03
Métodos Numéricos

Coordinacion curso:

Principal: Ing. Daniel Felipe Rodriguez Ramirez

Estudiantes/Autores

Nicolás Godoy <ngodoya@unal.edu.co> <1013113408>

Sofia Gonzalez <lagonzalezvi@unal.edu.co> <1011098142>

11 de mayo de 2026

Introducción y Motivación

El siguiente taller esta realizado en formato $\text{\LaTeX}$ para mayor comodidad a la hora de presentar el procedimiento en cada ejercicio para llegar a su respectiva solución. La resolución de los códigos de los ejercicios se realizó en Python y, en los casos en los que se requería, se utilizó Excel; en el presente documento se explican los códigos utilizados; sin embargo, para mayor profundidad en ellos, en los anexos se incluye el cuadernillo de Jupyter [Kluyver et al., 2016] o la respectiva hoja de cálculo, donde se mostraran los códigos con mayor profundidad. Aquí se realiza la explicación y la idea de como se llega a la solución mostrada en el documento. La regresión logística es uno de los modelos más usados para resolver problemas o situaciones que siguen una distribución de Bernoulli, más conocidos como problemas de clasificación binaria: situaciones en las que, a partir de ciertas variables medibles, se quiere estimar la probabilidad de que ocurra o no un evento (sí/no, pasa/no pasa, 0/1, éxito/fracaso). Estos problemas son bastante comunes en la estadística y su estructura matemática siempre es la misma, independientemente de su dominio: se tienen variables explicativas y una variable de respuesta binaria, y el objetivo es ajustar los parámetros de un modelo probabilístico que explique de la mejor manera posible los datos observados.

Basado en el interés de ambos integrantes en la estadística, en el presente proyecto se aborda esa tarea de ajuste/calibre de los parámetros de un modelo de regresión logística, utilizando un conjunto de datos sintético generado a partir de una relación logística conocida de antemano. Estos datos sintéticos fueron generados con base en una relación conocida (en este caso una ecuación), por lo que es posible verificar con precisión qué tan bien cada método recupera los parámetros buscados.

El ajuste de los parámetros de este tipo de modelos no tiene una solución algebraica cerrada, es decir, no existe una fórmula que dé directamente los valores óptimos, por lo que se hace necesario recurrir a un método iterativo, como ocurre en la cotidianidad.

Entre los métodos estudiados en el curso se encuentran el descenso del gradiente y el método de Newton-Raphson, los cuales, a pesar de tener diferencias entre sí, pueden aplicarse al mismo tipo de problema; el primero corresponde a un método de primer orden, en donde cada iteración se mueve en la dirección opuesta al gradiente de la función de costo, usando un tamaño de paso fijo (la tasa de aprendizaje). El segundo, por su parte, es un método de segundo orden que, además del gradiente, usa también la curvatura de la función (la matriz hessiana) para dar un paso un poco más acertado en cada iteración. Por su parte, la librería scikit ofrece distintas herramientas para la optimización de parámetros en problemas de clasificación binaria, por ejemplo, modelos ideales como regresión logística (LogisticRegression).

La comparación de ambos métodos, aplicados exactamente al mismo problema, permite comparar un aspecto esencial en la escogencia de un método para una optimización numérica: el costo computacional de cada iteración frente a la velocidad con la que el método converge o se acerca a la solución. La regresión logística no se limita únicamente al ámbito educativo o al rendimiento académico de estudiantes; su fundamentación matemática como modelo de respuesta binaria la convierte en una herramienta fundamental en múltiples disciplinas. Por ejemplo, en el campo de la salud y la epidemiología, permite estimar la probabilidad de que un paciente desarrolle o presente una determinada enfermedad (p. ej., presencia o ausencia de diabetes, riesgo de infarto) a partir de covariables biomédicas como la presión arterial, la edad o los niveles de glucosa. De manera análoga, en ingeniería de calidad predice la probabilidad de falla en una pieza industrial, y en finanzas evalúa el riesgo de impago de un crédito. Esta transversalidad convierte el estudio numérico de la calibración de sus parámetros en un problema relevante para la práctica profesional.

1. Planteamiento del problema

1.1. Contexto y pregunta de investigación

El problema abordado consiste en calibrar los parámetros de un modelo de regresión logística a partir de un conjunto de datos observados usando dos métodos numéricos distintos. La estructura del problema consiste en dos variables explicativas continuas y una respuesta binaria, esta se asemeja a la de muchos problemas de clasificación en ingeniería, por ejemplo, el decidir si un lote de producto pasa o no un control de calidad en función de variables ligadas al proceso

de este, además de un sinnúmero de aplicaciones en otros ámbitos, incluso en la misma estadística; por esta aplicabilidad en el mundo real, es que es un caso de estudio óptimo para el proyecto. Los datos utilizados son sintéticos, es decir, que no representan estudiantes reales, sino que son un banco de pruebas controlado en el que se conoce la relación entre las variables; esto permite verificar si cada algoritmo recupera correctamente esa relación.

La pregunta planteada para este proyecto en específico es: ¿Cómo se comparan el descenso del gradiente y el método de Newton-Raphson en términos de velocidad de convergencia, estabilidad numérica y precisión del modelo resultante, al enfrentarse a resolver el mismo problema de optimización?

1.2. Objetivos

Objetivo general: Comparar el desempeño de dos métodos, descenso del gradiente y Newton-Raphson, en la optimización de los parámetros de un modelo de regresión logística aplicado a un modelo estadístico de Bernoulli, donde la variable buscada es dicotómica (solo vive en fracaso o éxito).

Objetivos específicos:

- Formular el ajuste de un modelo de regresión logística como un problema de optimización sin restricciones.
- Implementar ambos métodos numéricos de forma modular y reproducible, con sus respectivos criterios de parada.
- Validar y comparar los resultados obtenidos frente a una implementación de referencia, en este caso, Scikit-learn.
- Comparar cuantitativamente la velocidad de convergencia, el costo computacional, la estabilidad numérica y la calidad de la clasificación resultante; con esto, ser capaz de decidir qué modelo es más adecuado para la tarea propuesta.

1.3. Alcance y limitaciones del proyecto

El proyecto se limita a un problema de clasificación binaria con dos variables explicativas: horas de estudio que un estudiante invierte y puntaje previo obtenido. Se trabaja sobre un conjunto de datos sintéticos de 200 observaciones, generado con una relación logística conocida y una semilla aleatoria fija (42), lo que garantiza reproducibilidad. Quedan fuera del alcance las variantes con regularización, Ridge o Lasso, los métodos de segundo orden aproximados (quasi-Newton, como BFGS o L-BFGS) más allá de como referencia de validación, y el caso multiclase.

1.4. Variables y restricciones

- **Variables explicativas:** x_1 = horas de estudio dedicadas a la asignatura (rango $[1, 10]$); x_2 = promedio de calificaciones previamente obtenidas por el estudiante (rango $[50, 100]$).
- **Variable de respuesta:** $y \in \{0, 1\}$, donde $y = 1$ significa que el estudiante aprueba la materia y $y = 0$ que la reprueba o pierde.
- **Parámetros del modelo:** el intercepto b y los coeficientes w_1, w_2 , que deben ser estimados a partir de los datos observados.

Los dos métodos fueron implementados desde cero usando NumPy. La librería Scikit-learn se usó únicamente como una solución de referencia para validar los resultados, y no como método propio del proyecto.

1.5. Criterios de evaluación de la solución

Existieron varios criterios usados para la evaluación de las soluciones: primero, la exactitud, precisión, exhaustividad o recall y F1-score que presentaba el conjunto de datos obtenido. Segundo, pérdida o log de entropía cruzada binaria, la cual se usó como medida directa de la calidad del ajuste probabilístico; tercero, número de iteraciones hasta la

convergencia de cada método, o hasta que se agotara el máximo permitido, y tiempo de cómputo empleado; y por último, la cercanía de los parámetros estimados respecto a la solución brindada por Scikit-learn.

1.6. Justificación de uso de estas métricas

La log-pérdida es la función de costo que ambos métodos minimizan (sección 3); compararla mide directamente qué tan bien resolvió cada método el problema de optimización en sí. Por otro lado, estas métricas de clasificación anteriormente mencionadas, fueron escogidas porque el conjunto de datos arrojado presenta un desbalance (78 % de la clase aprobó y 22 % no lo hizo), en este escenario evaluar solamente la exactitud puede ser engañoso, pues si el modelo siempre predice aprobado acertaría el 78 % de las veces sin haber aprendido nada. Por esto, se añadieron los criterios de precisión y el recall, que sí distinguen ese comportamiento, que es justamente el comportamiento exhibido por el resultado del descenso del gradiente en la sección 7.5.

2. Fundamentación y modelo matemático

El modelo de regresión logística calcula la probabilidad de que la variable de respuesta y sea igual a 1, dado un conjunto de variables explicativas $\mathbf{x}$. Para eso, se usa la función sigmoide (función que transforma cualquier número real en un valor entre 0 y 1) sobre una combinación lineal de los parámetros:

$$P(y = 1|\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = b + w_1 x_1 + w_2 x_2 \quad (1)$$

Donde b corresponde al intercepto, w_1 y w_2 son los pesos asociados a las horas de estudio (x_1) y al puntaje previo (x_2). Y como la sigmoide σ convierte cualquier número real en un valor entre 0 y 1, es fundamental para interpretar el resultado como una probabilidad.

Los parámetros (b y w_i) se ajustan usando el método de máxima verosimilitud, que busca que los datos observados sean lo más cercanos posible a los predichos. Como no se maximiza la verosimilitud directamente, se minimiza el negativo de su logaritmo, que es la función de costo que en este caso llamamos entropía cruzada binaria; en esta, m corresponde al número total de observaciones o muestras que se tiene en el conjunto de datos ($m = 200$):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad \hat{y}_i = \sigma(z_i) \quad (2)$$

Esta función se utiliza para calcular el error promedio de la muestra; es convexa con respecto a θ (los parámetros), lo que significa que cualquier mínimo local que encuentre un método de optimización es también el mínimo global. Para aplicar los dos métodos numéricos que usamos en este proyecto, necesitamos el gradiente y la matriz Hessiana de $J(\theta)$, que tienen expresiones bastante directas:

$$\nabla J(\theta) = \frac{1}{m} X^T (\hat{\mathbf{y}} - \mathbf{y}) \quad (3)$$

$$H(\theta) = \frac{1}{m} X^T W X, \quad W = \text{diag}(\hat{y}_i(1 - \hat{y}_i)) \quad (4)$$

Donde X es la matriz de diseño (con una columna adicional de unos para el intercepto) y W es una matriz diagonal con valores $\hat{y}_i(1 - \hat{y}_i)$, que siempre son no negativos. Lo que asegura que la Hessiana sea semidefinida positiva, así, a su vez, la función de costo es convexa.

2.1. Supuestos del modelo

Con la finalidad de simplificar el trabajo, se supone que las observaciones encontradas son independientes entre sí, que la relación entre las variables explicativas y el logit de la probabilidad de éxito es lineal y que no hay colinealidad perfecta entre las variables explicativas.

2.2. Generación de datos

Se generó un conjunto de datos sintético de $m = 200$ observaciones con una relación logística conocida de antemano. Las horas de estudio se generaron a partir de una distribución uniforme en $[1, 10]$, y el puntaje previo a partir de una distribución uniforme en $[50, 100]$. El logit real usado para generar los datos fue:

$$z = -8 + 0,8x_1 + 0,08x_2 \quad (5)$$

El -8 representa una probabilidad base baja de aprobar sin estudiar (porque existe el caso) y sin historial previo, mientras que los coeficientes $0,8$ y $0,08$ reflejan el peso significativo que ambas variables terminan teniendo en el resultado; los números se explican dado que el rango del puntaje previo (50 puntos) es también 10 veces mayor que el de las horas de estudio (9 horas).

La etiqueta binaria de cada observación se generó mediante un muestreo Bernoulli con probabilidad $\sigma(z_i)$, usando la semilla 42 de NumPy para poder garantizar reproducibilidad. El conjunto resultante contiene 156 observaciones de la clase 1 (78 %) y 44 de la clase 0 (22 %), este desbalance de aprobaciones se tiene en cuenta en la sección de análisis de resultados.

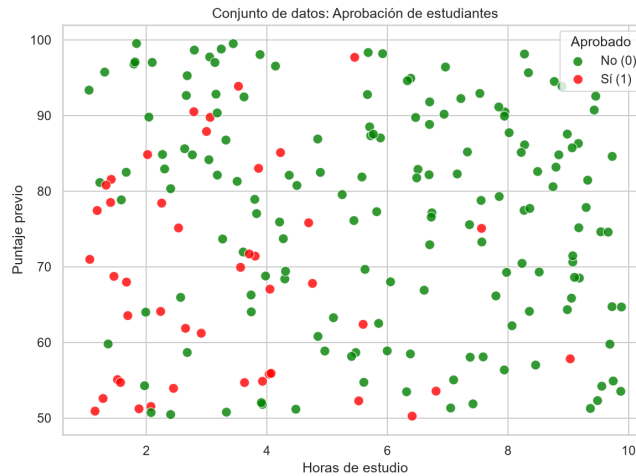


Figura 1: Conjunto de datos sintético de 200 observaciones

3. Métodos seleccionados

Como se comentó anteriormente, se seleccionaron dos métodos iterativos de optimización sin restricciones, ambos aplicados directamente sobre la función de costo $J(\theta)$. Estos fueron seleccionados porque representan dos temas recurrentes en el curso: un método de primer orden y uno de segundo, su oposición interna permite ilustrar el compromiso central de la optimización numérica: costo por iteración frente a velocidad de convergencia. Parten del mismo punto inicial ($\theta = 0$) y del mismo criterio de parada por norma del gradiente, por lo que sus trayectorias de convergencia son comparables.

3.1. Descenso del gradiente: primer método

Es un método iterativo de optimización de primer orden que actualiza los parámetros en dirección del máximo decrecimiento de la función de costo, es decir, dirección opuesta al gradiente. Esta actualización sigue la regla:

$$\theta_{k+1} = \theta_k - \alpha \cdot \nabla J(\theta_k) \quad (6)$$

Entonces, α es la tasa de aprendizaje, un número positivo que controla qué tan grande es el paso que damos en la dirección del gradiente; si α es muy pequeño, el método avanza muy lento y se necesitan muchas iteraciones, por el contrario, si es muy grande, se puede llegar a “saltar” el mínimo y el método puede oscilar o incluso divergir.

Este es un método de primer orden, pues solo usa información de la primera derivada (el gradiente), tiene convergencia lineal: el error se reduce en una fracción constante en cada iteración. Esto significa que, si se desea obtener una solución muy precisa, se necesitan bastantes iteraciones; además, la velocidad de convergencia depende de qué tan bien se elija α en relación con la curvatura de la función de costo (la cual es determinada por la segunda derivada o Hessiana), por lo que, en la práctica, el método es sensible a la escala de las variables y al valor de α .

Se detiene cuando la norma del gradiente es menor que $\text{tol} = 1 \times 10^{-6}$, o al alcanzar la iteración 2000, con una $\alpha = 0,01$, un valor típico en la literatura para este tipo de problemas, aunque resultó insuficiente para lograr convergencia estable dado el desbalance de escalas entre variables.

El algoritmo implementado fue el siguiente:

1. Construir la matriz de diseño X añadiendo una columna de unos para el intercepto.
2. Inicializar $\theta = 0$.
3. Para cada iteración: calcular $\hat{y} = \sigma(X\theta)$, el gradiente $\nabla J(\theta)$, y actualizar θ , registrando la pérdida en el historial.
4. Detener el ciclo si $\|\nabla J(\theta)\| < \text{tol}$; en caso contrario, continuar hasta la máxima iteración permitida.

3.2. Método de Newton-Raphson

Es un método iterativo de optimización de segundo orden que usa tanto el gradiente como la matriz Hessiana de la función de costo para construir una aproximación cuadrática local de $J(\theta)$ en cada iteración y salta directamente al mínimo de esa aproximación. Sigue la regla:

$$\theta_{k+1} = \theta_k - H(\theta_k)^{-1} \cdot \nabla J(\theta_k) \quad (7)$$

Aquí, en vez de usar una tasa de aprendizaje fija, se calcula el tamaño de paso óptimo en cada iteración usando la curvatura de la función (la matriz Hessiana). En la práctica, en vez de invertir directamente la Hessiana se resuelve el sistema lineal:

$$H(\theta_k)\Delta\theta = \nabla J(\theta_k) \quad (8)$$

para encontrar el paso $\Delta\theta$.

Su convergencia es cuadrática cerca del óptimo, pues el número de cifras decimales correctas se duplica aproximadamente en cada iteración. En problemas que tienen un comportamiento bueno, este método converge en pocas iteraciones (solo 7 en este caso); sin embargo, la desventaja es que cada iteración es más costosa computacionalmente, porque se debe construir y resolver un sistema lineal con la Hessiana.

La función debe ser dos veces diferenciable, con Hessiana definida, esta Hessiana debe ser invertible en cada iteración; en este caso, para evitar un mal comportamiento cuando las probabilidades predichas sean cercanas a 0 o 1, se añadió una regularización de Tikhonov ($H + \varepsilon I$, con $\varepsilon = 1 \times 10^{-6}$) antes de resolver el sistema. Y, por último, el punto inicial debe estar cerca del óptimo para así garantizar la convergencia cuadrática; en problemas convexos suele bastar partir desde 0.

El algoritmo implementado fue el siguiente:

1. Construir la matriz de diseño X añadiendo una columna de unos para el intercepto.
2. Inicializar $\theta = 0$.
3. Para cada iteración: calcular $\hat{y} = \sigma(X\theta)$, el gradiente g , la matriz W y la Hessiana H , y actualizar θ .
4. Detener el ciclo si $\|g\| < \text{tol}$ (la misma que en el método anterior); en caso contrario, continuar hasta la máxima iteración permitida (50).

Se escogió este método como contraparte de segundo orden pues, en un problema como este (baja dimensionalidad, tres parámetros), el costo de resolver un sistema 3×3 en cada iteración es insignificante frente a la ganancia en la velocidad de convergencia; especialmente al contrastarlo con el descenso del gradiente, cuya ventaja existe en problemas de alta dimensión donde construir la Hessiana es inviable, mientras que acá simplemente no converge.

4. Implementación Computacional

La arquitectura computacional del proyecto fue diseñada siguiendo principios de modularidad, reusabilidad y código limpio dentro del ecosistema de Python 3. El propósito principal de la librería PyStatistics es proveer una implementación transparente e independiente de los algoritmos de optimización para regresión logística, permitiendo un control preciso sobre las métricas internas de entrenamiento, la tolerancia numérica y el análisis de convergencia.

4.1. Estructura del Proyecto y Módulos

El código fuente se organizó bajo una estructura de paquete en el directorio `src/`, separando las responsabilidades de optimización, modelado y evaluación:

- `src/optimizers/optimizer.py`: Define la interfaz y clase base abstracta `Optimizer`, la cual estandariza el contrato para los métodos de optimización (definiendo métodos como `fit` y `get_history`).
- `src/optimizers/gradient_descent.py`: Contiene la clase `GradientDescent`, encargada de ejecutar la optimización de primer orden mediante el cálculo vectorial del gradiente sobre la función de costo *Log-Loss*.
- `src/optimizers/newton_raphson.py`: Implementa la clase `NewtonRaphson`, la cual ejecuta la optimización de segundo orden incorporando el cálculo e inversión numérica de la matriz Hessiana para lograr una convergencia cuadrática.
- `src/models/logistic_regression.py`: Modulo principal que define la clase `LogisticRegression`. Administra la transformación sigmoide, la adición del término de sesgo (*bias/intercept*), la coordinación con el optimizador seleccionado y las funciones de predicción probabilística (`predict_proba`) y binaria (`predict`).

4.2. Entorno Computacional y Bibliotecas

Para el desarrollo y soporte operativo de la solución se emplearon las siguientes bibliotecas especializadas:

- **NumPy**: Empleada para el procesamiento matricial y operaciones algebraicas vectorizadas (productos punto, transposiciones e inversión de matrices), asegurando la eficiencia en la ejecución.
- **Pandas**: Utilizada en el estructurado y consolidado tabular de las métricas de desempeño y resultados comparativos.
- **Matplotlib y Seaborn**: Empleadas para la generación de gráficos técnicos, incluyendo la superficie de la función sigmoide, las fronteras de decisión y las matrices de confusión.
- **Scikit-Learn**: Utilizada exclusivamente como *baseline* o modelo de referencia para la validación numérica de la implementación propia, empleando la clase `LogisticRegression` con el parámetro de regularización desactivado (`C=np.inf`) y el optimizador `solver='lbfgs'`.

4.3. Decisiones de Implementación y Estabilidad Numérica

Durante el desarrollo se tomaron decisiones clave para garantizar la estabilidad aritmética de los métodos:

1. **Prevención de desbordamiento en la sigmoide**: Para evitar errores de tipo *overflow* o *underflow* al calcular e^{-z} con valores extremos de z , se implementó un truncamiento numérico (*clipping*) sobre el rango $[-50, 50]$.

2. **Estabilidad en la Log-Loss:** En el cálculo del logaritmo de las probabilidades ($\log(p)$ y $\log(1 - p)$), se añadió una constante infinitesimal $\epsilon = 10^{-15}$ para evitar indefiniciones matemáticas ($\log(0)$).
3. **Inversión de la Matriz Hessiana:** En la clase `NewtonRaphson`, el cálculo del paso $\mathbf{H}^{-1} \nabla J(\boldsymbol{\theta})$ se resolvió mediante el solucionador de sistemas lineales de NumPy (`np.linalg.solve`) en lugar de la inversión explícita directa, mejorando la precisión y la velocidad computacional.

4.4. Procedimiento de Verificación y Validación

La validez del código se confirmó mediante un protocolo de dos etapas:

- **Pruebas de consistencia interna:** Se verificó que la función de costo disminuyera monótonamente en cada iteración del historial (`history`) y que las probabilidades predichas se mantuvieran estrictamente acotadas en el intervalo $[0, 1]$.
- **Validación cruzada contra Scikit-Learn:** Se contrastaron los vectores de coeficientes ($\mathbf{w}$), el intercepto (b) y la función de pérdida final obtenidos por `PyStatistics` (Newton-Raphson) frente a la solución de `Scikit-Learn` (L-BFGS), verificando una coincidencia numérica hasta el tercer decimal.

5. Diseño de pruebas y experimentos

El experimento consistió en entrenar el mismo modelo de regresión logística sobre el mismo conjunto de 200 observaciones, utilizando tres configuraciones de optimización: descenso del gradiente, con $\alpha = 0,01$ y un máximo de iteración de 2000, con una tolerancia de 1×10^{-6} . Newton-Raphson, con un máximo de iteraciones de 50 y misma tolerancia. Finalmente, Scikit-learn (L-BFGS), como solución de referencia de alta confiabilidad del proyecto.

Para ambas configuraciones se midieron las siguientes métricas sobre el conjunto completo de datos. Cabe aclarar que no se realizó una partición de entrenamiento o prueba, pues el objetivo es comparar la optimización numérica y no evaluar la capacidad de generalización del modelo, como fue definido como fuera del alcance en la sección 2.3:

- Número de iteraciones ejecutadas hasta la convergencia o hasta agotar el máximo permitido.
- Tiempo de cómputo total del ajuste.
- Exactitud, precisión, exhaustividad y F1-score de la clasificación resultante.
- Log-pérdida (entropía cruzada binaria) sobre las probabilidades predichas.
- Valor final del intercepto y los coeficientes, contrastados con el valor real ($b = -8$, $w_1 = 0,8$, $w_2 = 0,08$).

Adicionalmente, se registró el historial completo de la función de pérdida en cada iteración de ambos métodos, esto permite construir las curvas de convergencia y evaluar visualmente la estabilidad de cada trayectoria (sección 7.3.)

6. Resultados

Después de evaluar ambos modelos, se obtuvieron los siguientes resultados organizados en tablas:

6.1. Tabla comparativa de métricas

Método	Iteraciones	Tiempo (s)	Accuracy	Precisión	Recall	F1-score	Log-Loss
Descenso del gradiente	2000	0,03867	0,78	0,780	1,0000	0,8764	1,1489
Newton-Raphson	7	0,00067	0,81	0,843	0,9295	0,8841	0,3925
Scikit-learn	26	0,01479	0,81	0,843	0,9295	0,8841	0,3925

Cuadro 1: Métricas de desempeño de los métodos sobre el conjunto de 200 observaciones.

6.2. Parámetros estimados

Método	Intercepto (b)	w_1 (horas de estudio)	w_2 (puntaje previo)
Valor real (tomado de la generación)	-8	0,8000	0,0800
Descenso del gradiente	-0,5260	1,3123	0,0202
Newton-Raphson	-4,8466	0,5220	0,0516
Scikit-learn	-4,8464	0,5220	0,0516

Cuadro 2: Comparación de los parámetros estimados por cada método frente al valor real usado para generar los datos.

Newton-Raphson y scikit-learn coinciden hasta la tercera cifra decimal en los tres parámetros, ambos aproximándose razonablemente al valor real que fue generado, teniendo en cuenta que solo se hicieron 200 observaciones y existe ruido de muestreo. Por otro lado, el descenso del gradiente se aleja de manera considerable de ambos valores; esto indica que no alcanzó la convergencia.

6.3. Explicación de la diferencia de iteraciones: Newton-Raphson vs. Scikit-Learn

Se observa una diferencia representativa entre las 7 iteraciones empleadas por nuestra implementación de Newton-Raphson y las 26 iteraciones requeridas por Scikit-Learn (`solver='lbfgs'`). Esta disparidad no implica un menor desempeño por parte de Scikit-Learn, sino una diferencia conceptual en sus algoritmos de optimización:

- **Newton-Raphson Puro (Segundo Orden Exacto):** Construye e invierte la matriz Hessiana exacta ($\mathbf{H} \in \mathbb{R}^{3 \times 3}$) en cada iteración. Al disponer de la curvatura exacta, logra una velocidad de convergencia cuadrática, requiriendo únicamente 7 iteraciones para alcanzar el mínimo de la función de costo.
- **Scikit-Learn (L-BFGS - Cuasi-Newton de Memoria Limitada):** No calcula ni invierte la matriz Hessiana real directamente. En su lugar, construye una aproximación iterativa de la curvatura utilizando únicamente la información de las diferencias de los gradientes de iteraciones previas. Como consecuencia de trabajar con una aproximación de la Hessiana, requiere un mayor número de pasos (26 iteraciones) para lograr la misma precisión final. Sin embargo, reduce drásticamente la carga computacional por iteración en problemas de alta dimensión.

Cuadro 3: Comparativa integral de desempeño, complejidad y vulnerabilidades entre los tres métodos de optimización.

Criterio / Método	Descenso del Gradiente (PyStatistics)	Newton-Raphson (PyStatistics)	Scikit-Learn (L-BFGS)
Orden de Optimización	Primer orden (utiliza solo el gradiente $\nabla J(\theta)$).	Segundo orden exacto (utiliza gradiente $\nabla J(\theta)$ y Hessiana $\mathbf{H}$).	Cuasi-Newton (aproxima la Hessiana de forma iterativa).
Iteraciones Ejecutadas	2000 (agota el máximo permitido sin converger).	7 (convergencia cuadrática).	26 (convergencia superlineal).
Tiempo de Cómputo (s)	0,03867 s.	0,00067 s (el más rápido para este conjunto).	0,01479 s.
Log-Loss Final / Accuracy	Log-Loss: 1,1489 Accuracy: 0,78.	Log-Loss: 0,3925 Accuracy: 0,81.	Log-Loss: 0,3925 Accuracy: 0,81.
Costo por Iteración	$O(m \cdot p)$ Muy bajo. Operaciones vectoriales simples.	$O(m \cdot p^2 + p^3)$ Extremadamente alto por la inversión/resolución de $\mathbf{H}$.	$O(m \cdot p + m \cdot k)$ Moderado-Bajo. Guarda solo los últimos k gradientes.
Peligros y Sensibilidades	- Alta sensibilidad a la tasa de aprendizaje (α). - Falla grave ante variables sin escalamiento/normalización.	- Inviabile en alta dimensión ($p > 10,000$ variables) por almacenamiento y cómputo de $\mathbf{H}$.	- Ligeramente dependiente del parámetro de memoria k guardado.
Casos de Falla y Vulnerabilidad	- Oscilación e inestabilidad (zigzag constante). - Clasificación sesgada (predijo el 100 % a favor de una sola clase).	- Mal funcionamiento si la Hessiana es singular ($\det(\mathbf{H}) \approx 0$) o mal condicionada.	- Pérdida de precisión si la función de pérdida no es suave o presenta bastante ruido.

6.4. Curvas de convergencia

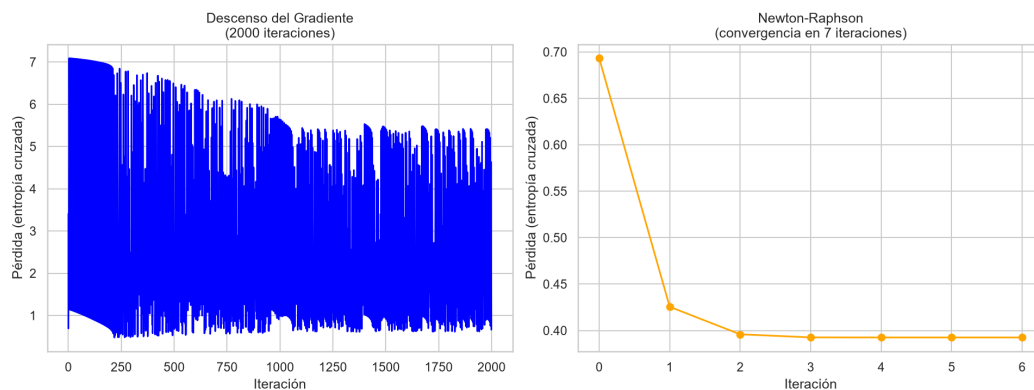


Figura 2: Evolución de la función de pérdida en cada iteración para el descenso del gradiente (izquierda) y Newton-Raphson (derecha). Nótese la diferencia de escala en el eje horizontal.

Sin entrar en detalles, se observa una gran diferencia en el comportamiento de ambos métodos. Esto es aún más evidente en la siguiente gráfica, donde se juntan ambas convergencias en la misma escala:

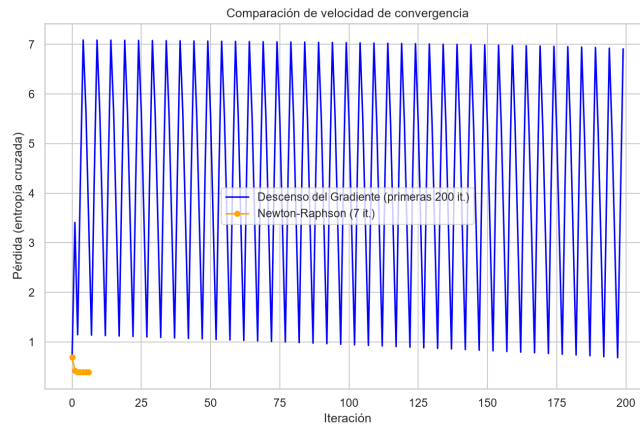


Figura 3: Comparación directa de la velocidad de convergencia en las mismas unidades de pérdida.

6.5. Fronteras de decisión

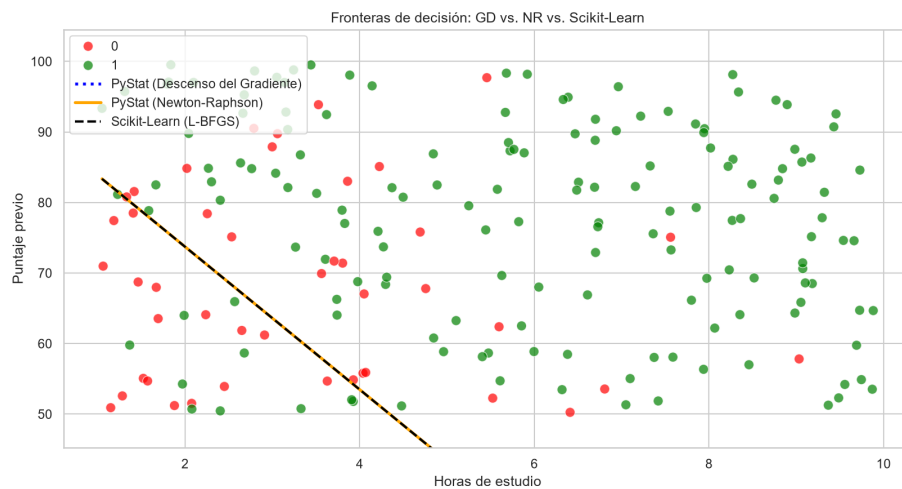


Figura 4: Fronteras de decisión obtenidas por los tres métodos, superpuestas sobre el conjunto de datos. Nótese que, en el rango tomado, la encontrada por el descenso del gradiente ni siquiera es visible.

La frontera obtenida por Newton-Raphson es visualmente indistinguible de la de Scikit-learn, mientras que, la obtenida por el descenso del gradiente ni siquiera es visible en la figura, pues sus valores son tan alejados que cae en valores de puntaje previos entre -3 y -50 para el rango de horas de estudio graficado, lo que no solo es bastante alejado del eje visible, sino que no tiene sentido físico; esto, como se verá posteriormente, termina clasificando a todos los estudiantes como aprobados sin distinguir realmente cuáles o hicieron o no (Figura 4.)

6.6. Matrices de confusión

Una matriz de confusión es una tabla que muestra los aciertos y los errores de los resultados de un algoritmo clasificador. En este caso, las obtenidos para el proyecto fueron:

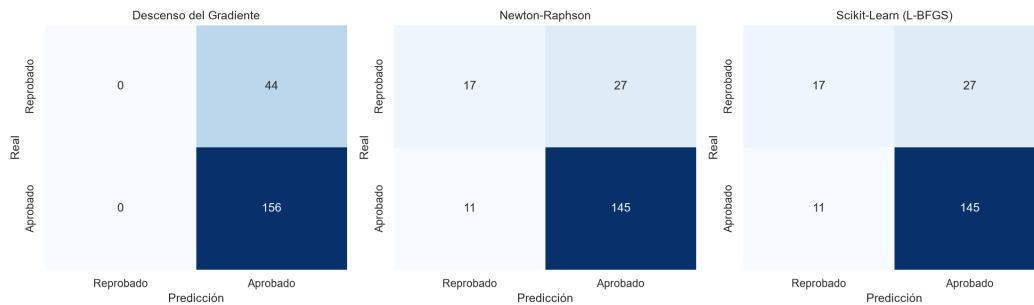


Figura 5: Matrices de confusión de los tres métodos sobre el conjunto de 200 observaciones.

La matriz de confusión del descenso del gradiente muestra que el método clasifica correctamente a todos los estudiantes en 1 ($\text{recall}=1,0$), pero esto lo hace sin importar clasificar erróneamente a una fracción importante de los negativos (desaprobados) reales (con una precisión de 0,78), esto es consistente con el desplazamiento de la frontera al no haber convergido. Por otro lado, Newton y Scikit-learn producen matrices prácticamente idénticas y un mejor balance entre falsos negativos y falsos positivos.

7. Análisis comparativo entre ambos métodos

7.1. Precisión y error

Newton-Raphson alcanza una log-loss de 0,3925, curiosamente idéntica hasta la cuarta cifra decimal a la de scikit-learn, y una exactitud de 0,81. El descenso del gradiente, después de agotar el máximo de iteraciones, alcanza un log-loss muy superior (por encima de 5 en sus últimas iteraciones) y una exactitud ligeramente menor (0,78). Esto evidencia que, con los parámetros usados, no logró aproximarse óptimamente de manera remotamente similar a como lo hizo un método de segundo orden.

7.2. Rapidez y costo computacional

En cuanto al número de iteraciones, Newton-Raphson requirió solo de 7, frente a las 2000 agotadas por el descenso del gradiente. Sin embargo, se debe tener en cuenta que el costo por iteración de Newton es mucho mayor, pues cada paso implica construir una Hessiana 3×3 y resolver un sistema lineal, mientras que cada paso en el gradiente solo requiere un producto matriz-vector. En este problema específico, este costo adicional es prácticamente insignificante frente a la velocidad de convergencia del método, resultando en un tiempo menor. Sin embargo, es un factor a tener en cuenta en problemas de alta dimensionalidad, donde construir y resolver un sistema con el Hessiano puede ser engorroso o complejo, en estas situaciones, el bajo costo del descenso del gradiente lo hace la opción óptima para la resolución.

7.3. Convergencia y estabilidad

La figura 2 muestra el contraste entre ambos métodos, la curva de Newton-Raphson muestra una caída rápida y monótona de la pérdida, que se estabiliza cerca de 0,4 después de 7 iteraciones. Por otro lado, la curva del descenso del gradiente no muestra una tendencia sostenida: inicialmente desciende, pero luego oscila entre valores muy altos y muy bajos. Este comportamiento no parece atenuarse a medida que se acerca a las 2000 iteraciones, lo que indica no convergencia; además, evidencia que el tamaño de paso seleccionado es demasiado grande para la dirección de mayor escala (puntaje previo) y demasiado pequeño para la de menor escala (horas de estudio). Este comportamiento extraño del descenso del gradiente es común cuando las variables explicativas tienen escalas muy distintas y no son normalizadas antes.

7.4. Sensibilidad a parámetros e hiperparámetros

En este problema, NR es relativamente insensible al punto de partida y no requiere ajustar ningún hiperparámetro de tamaño de paso; a diferencia del DDG, el cual es bastante sensible a la elección de α : si se escogiese un valor más pequeño, eliminaría la oscilación, pero requeriría más iteraciones, mientras que un valor más grande provocaría divergencia. Ya sensible de por sí, es agravada por la diferencia de escala entre las variables (de aproximadamente un factor de 10 entre ambas), que hace que no exista un único α adecuado para ambas direcciones sin antes normalizar los datos.

8. Discusión

Los resultados son consistentes con la teoría de optimización numérica, que establece que, para funciones convexas y de baja dimensionalidad, los métodos de segundo orden como Newton-Raphson aprovechan la información de curvatura para converger en menos iteraciones, esto a costa de una iteración más robusta y engorrosa que, en este caso, resulta insignificante cuando el número de parámetros es pequeño. El resultado más evidente es el fallo del descenso del gradiente; sin embargo, se destaca que este no falló por un error de implementación, sino por una limitación importante del método: su sensibilidad al escalamiento relativo de las variables. Esto sugiere que, si se repitiese el experimento con las variables normalizadas, este podría lograr converger de forma estable con la misma tasa de aprendizaje, lo que es fuertemente apoyado por el equipo de trabajo como trabajo futuro.

Además, una limitación adicional es que el desempeño de cada método dependió directamente del conjunto de datos usado para entrenar el modelo, sin una partición de validación independiente o de prueba; lo cual es adecuado para el objetivo del proyecto, que solo se enfoca en comparar las optimizaciones y no en la capacidad de generalización del modelo; sin embargo, esto implica que la exactitud de 0,81 que se reporta no puede interpretarse como una estimación de lo que pasaría si el modelo se usase con estudiantes reales o datos distintos a los usados. Otra limitación es que el problema solo tiene tres parámetros, lo que significa una ventaja casi garantizada para NR, como se mencionó antes, si los parámetros aumentasen, la rigurosidad de Newton sería muchísimo más determinante en qué método escoger.

9. Conclusiones y recomendaciones

En respuesta al objetivo general planteado, se concluye que, para el caso estudiado (tres parámetros, 200 observaciones, variables sin normalizar), el método de Newton-Raphson es claramente superior al Descenso del gradiente: converge en 7 iteraciones frente a las 2000 agotadas por el segundo, alcanza una log-loss y una exactitud prácticamente idénticas a las de Scikit-learn, y su costo adicional es sopesado por esta misma velocidad de convergencia, dado la baja dimensionalidad del problema; es el método más robusto, preciso y rápido, sin desventajas prácticas relevantes. Por otro lado, el Descenso del Gradiente, usando un $\alpha = 0,01$, no logró converger de forma estable en el máximo de iteraciones permitido. Esto evidencia una limitación bastante importante de los métodos de primer orden con paso fijo: su desempeño depende mucho del preprocesamiento de los datos y del correcto ajuste de la tasa de aprendizaje, responsabilidad del usuario que implementa el algoritmo y no del algoritmo en sí. Implica que, si se desea que este método sea viable en problemas como este, se necesitan modificaciones adicionales, por ejemplo, normalizar las variables o lograr una tasa de aprendizaje adaptativa. Como recomendaciones para un trabajo futuro se propone: - Repetir el experimento normalizando las variables explicativas para verificar si el Descenso del gradiente logra converger de forma estable con la misma tasa de aprendizaje. - Explorar una tasa de aprendizaje adaptativa o variantes del método (momentum, Adam) que sean más robustas a diferencias de escala entre variables. - Extender la comparación a un problema donde exista una mayor dimensionalidad, donde la ventaja previa de NR se reduzca o incluso revierta, debido al costo de construir y resolver el Hessiano. - Para un caso real, incorporar un primer entrenamiento para evaluar la capacidad de generalización del modelo, para así poder complementar el análisis de optimización numérica.

10. Contribución de los integrantes

Sofía González: Redacción del documento escrito, planteamiento matemático, generación y análisis de las figuras.

Nicolás Godoy: Implementación y creación del algoritmo en su totalidad, implementación de los optimizadores GradientDescent y NewtonRaphson, redacción de la sección 5 (implementación computacional).

Ambos integrantes participaron en la revisión final del informe y en la preparación de la presentación tipo pitch para la sustentación del proyecto.

El código fuente completo (paquete `src/`, con los módulos `optimizar.py`, `gradient_descent.py`, `newton_raphson.py` y `logistic_regression.py`), junto con las instrucciones referentes al desarrollo y validación de los algoritmos, se encuentran disponibles en el repositorio oficial de GitHub y en la carpeta de soporte adjunta a esta entrega.

Instrucciones de ejecución:

Para reproducir completamente los resultados y experimentos presentados en este informe, siga los pasos a continuación desde la terminal o consola de comandos:

1. Clonar el repositorio oficial desde GitHub:

```
git clone https://github.com/ngodoya/Metodos-Numericos-Proyecto.git
cd Metodos-Numericos-Proyecto
```

2. Crear y activar un entorno virtual (recomendado):

```
python -m venv venv
# En Linux/macOS:
source venv/bin/activate
# En Windows (PowerShell):
.\venv\Scripts\Activate.ps1
```

3. Instalar las dependencias del proyecto:

```
pip install -r requirements.txt
```

4. Ejecutar el cuaderno interactivo de Jupyter:

```
jupyter notebook
```

Abra el archivo `notebooks/benchmarking.ipynb` o el cuadernillo principal para ejecutar secuencialmente todas las celdas de pruebas y generar las gráficas de soporte.

Bibliografía

[Kluyver et al., 2016] Kluyver, T. et al. (2016). Jupyter notebooks – a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press.